

Informe de Desempeno MPI Document Search Engine

Proyecto Final

1 Resumen

Este proyecto implementa un motor de busqueda paralelo con MPI basado en TF-IDF y similitud coseno. En esta version la indexacion se distribuye entre los procesos (TF-IDF e IDF en paralelo) y se reduce el volumen de datos transferidos al intercambiar solo el vocabulario global y los Top-K, lo cual mejora el rendimiento global. El informe presenta el algoritmo, la paralelizacion y un analisis detallado con las graficas obtenidas.

2 Conceptos clave (explicados en simple)

- **Corpus:** conjunto de documentos de texto usados como base de busqueda.
- **Vocabulario:** lista de palabras unicas presentes en el corpus.
- **Tokenizacion:** proceso de dividir el texto en palabras (tokens).
- **TF (Term Frequency):** frecuencia relativa de una palabra en un documento.
- **DF (Document Frequency):** numero de documentos que contienen una palabra.
- **IDF (Inverse Document Frequency):** peso que reduce palabras muy comunes.
- **TF-IDF:** producto $TF \times IDF$; representa importancia de una palabra.
- **Similitud coseno:** mide el angulo entre dos vectores; 1 significa muy similares.
- **Top-K:** los K documentos con mayor score de similitud.
- **Scatter/Gather/Bcast/Allreduce:** operaciones MPI para repartir, recolectar, difundir y reducir datos entre procesos.
- **Rank:** identificador de cada proceso MPI (0 es el master).

3 Generacion de corpus de prueba

Los corpus usados en las pruebas se generan con `generate_corpus.py`. El script crea un vocabulario sintetico de tamano configurable (por defecto 5000), genera documentos con una distribucion tipo Zipf (algunas palabras aparecen mucho mas que otras) y produce una consulta con un numero fijo de terminos. Esto permite crear conjuntos de 100, 500, 1000 y 5000 documentos de manera reproducible usando una semilla fija.

4 Descripción del algoritmo

Cada documento se convierte en un vector TF-IDF y la consulta también se convierte en un vector del mismo tamaño. Luego se calcula la similitud coseno entre la consulta y cada documento. Finalmente se ordenan los resultados y se devuelven los Top-K documentos más relevantes.

4.1 TF-IDF

Para cada término t en el documento d :

$$TF(t, d) = \frac{\text{freq}(t, d)}{\text{total de términos en } d}$$

$$IDF(t) = \log \left(\frac{N}{1 + df(t)} \right)$$

El vector final es $TF \cdot IDF(t, d) = TF(t, d) \cdot IDF(t)$, donde N es el número total de documentos y $df(t)$ es el número de documentos que contienen el término.

4.2 Similitud coseno

$$\text{sim}(q, d) = \frac{q \cdot d}{\|q\| \|d\|}$$

Los documentos se ordenan de mayor a menor según esta similitud.

5 Paralelización con MPI

La versión actual distribuye la indexación y reduce comunicación:

1. **Partición del corpus:** todos los ranks listan los archivos y cargan solo su subconjunto local.
2. **Vocabulario global:** cada rank crea un vocabulario local; el rank 0 los fusiona y lo difunde con `MPI_Bcast`.
3. **DF/IDF en paralelo:** cada rank calcula DF local y se combina con `MPI_Allreduce` para obtener DF global e IDF.
4. **TF-IDF local:** cada rank calcula TF-IDF solo para sus documentos.
5. **Top-K distribuido:** cada rank calcula similitudes y envía su Top-K; el master los fusiona con `MPI_Gatherv`.

Este flujo elimina el `Scatter` de la matriz TF-IDF completa y reduce el tráfico total a vocabulario y Top-K.

6 Pipeline del search_engine

El flujo completo del programa puede resumirse en los siguientes pasos:

1. **Lectura del corpus:** cada rank lista los archivos y carga solo su partición local en memoria.
2. **Vocabulario global:** se construye un vocabulario local y se fusiona en el master para producir el vocabulario global.

3. **Calculo de DF/IDF:** cada rank calcula DF local, se reduce para obtener DF global y se calcula IDF.
4. **Vectorizacion:** cada rank calcula TF-IDF para sus documentos y el vector TF-IDF de la consulta.
5. **Busqueda:** se calcula la similitud coseno para cada documento local y se obtiene el Top-K local.
6. **Recoleccion final:** el master recibe los Top-K locales, realiza el merge y entrega el ranking final.

Este pipeline esta alineado con la forma en la que se midieron los tiempos de indexacion, busqueda y merge en los resultados experimentales.

7 Mejora de rendimiento: TF-IDF/IDF en paralelo y menos datos transferidos

La ultima mejora se enfoco en reducir el tiempo de indexacion y el trafico MPI. Antes, el master calculaba el vocabulario, IDF y toda la matriz TF-IDF, y luego distribuia esa matriz completa con `MPI_Scatterv`. Esto generaba dos problemas: (1) una fase de indexacion totalmente serial en el master y (2) un volumen grande de datos enviados a todos los procesos.

En la version actual, el trabajo se distribuye desde el inicio:

- **Vocabulario paralelo:** cada rank construye su vocabulario local y el master los fusiona para crear el vocabulario global.
- **DF/IDF paralelos:** cada rank calcula DF local y se realiza un `MPI_Allreduce` para obtener DF global y calcular IDF en todos.
- **TF-IDF local:** cada rank calcula TF-IDF solo para sus documentos, eliminando la necesidad de enviar una matriz completa por la red.
- **Comunicacion reducida:** solo se transfieren el vocabulario global y los Top-K locales, lo que reduce significativamente el trafico MPI.

El impacto se observa en los resultados: el tiempo de **Scatter** es practicamente cero (Figura 4) y el tiempo total baja con P en corpus grandes (Figura 1), con speedup notable hasta 32 procesos (Figura 2). El nuevo cuello de botella se desplaza a la recoleccion de Top-K (**Gather/Merge**) cuando P es alto.

8 Configuracion experimental

Las pruebas se realizaron en el cluster usando los mismos parametros que el estudio anterior. Se evaluaron corpus de 100, 500, 1000 y 5000 documentos, con vocabulario de alrededor de 5000 palabras, Top-K = 10 y multiples repeticiones. Los resultados se almacenan en `results/`.

9 Resultados y analisis

9.1 Tiempo total

La Figura 1 muestra reducciones claras del tiempo total al aumentar procesos, especialmente para corpus grandes. Por ejemplo, para 5000 documentos el tiempo total baja de 45.0 s ($P=1$) a 11.5 s ($P=4$) y alcanza un minimo de 8.23 s en $P=32$. En corpus pequenos el overhead limita la mejora.

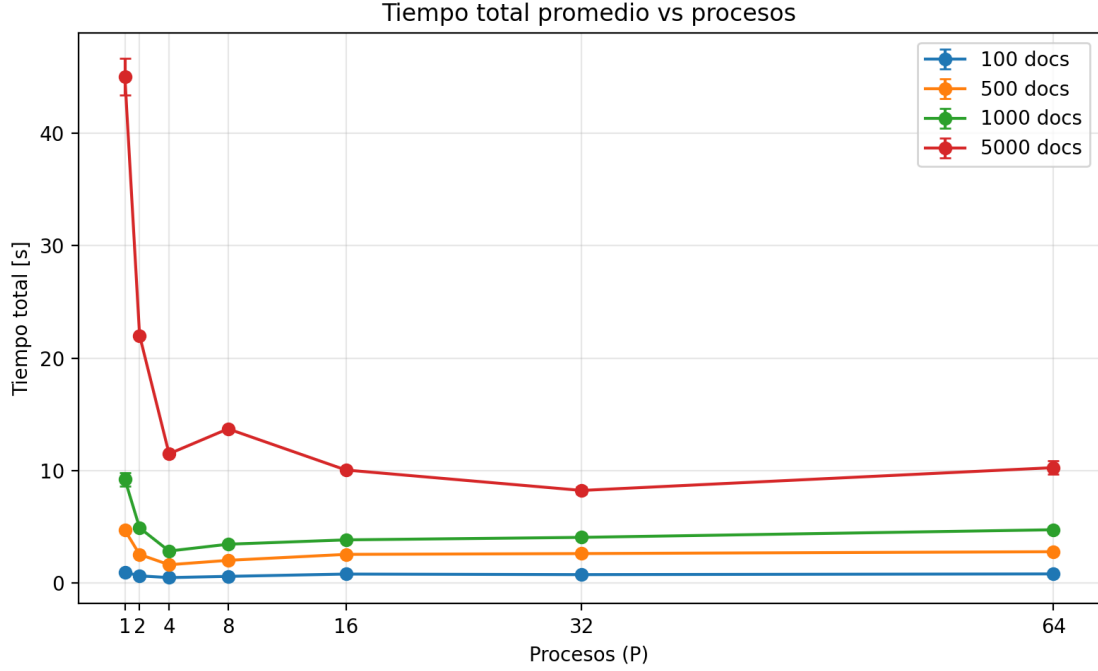


Figure 1: Tiempo total promedio vs procesos.

9.2 Speedup y eficiencia

La Figura 2 evidencia mejoras sustanciales: para 5000 documentos el speedup llega a 3.92 ($P=4$) y 5.47 ($P=32$). Para 1000 documentos el mejor speedup es 3.25 ($P=4$). La eficiencia es alta en $P=2-4$ para corpus grandes (cercana a 1), pero disminuye al aumentar P por sobrecarga de sincronizacion y recoleccion de resultados.

9.3 Desglose por fases

La Figura 4 muestra que **Scatter** desaparece (tiempo aproximadamente cero) al eliminar la distribucion de la matriz TF-IDF. La indexacion ahora se reparte entre procesos y disminuye con P . Sin embargo, la fase de **Gather/Merge** crece para P altos (por ejemplo en $P=8$ para 5000 documentos), convirtiendose en el principal limite de escalamiento.

9.4 Observaciones clave

- **Mejora global:** la indexacion paralela reduce el tiempo total de forma notable, especialmente en corpus grandes.

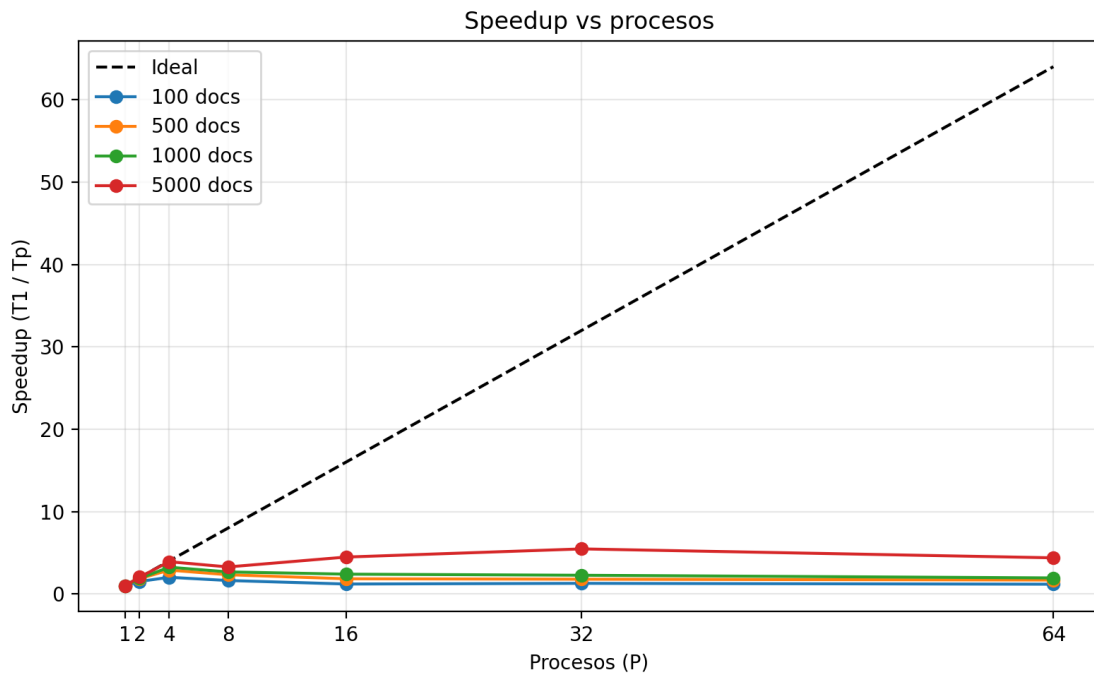


Figure 2: Speedup por cantidad de procesos.

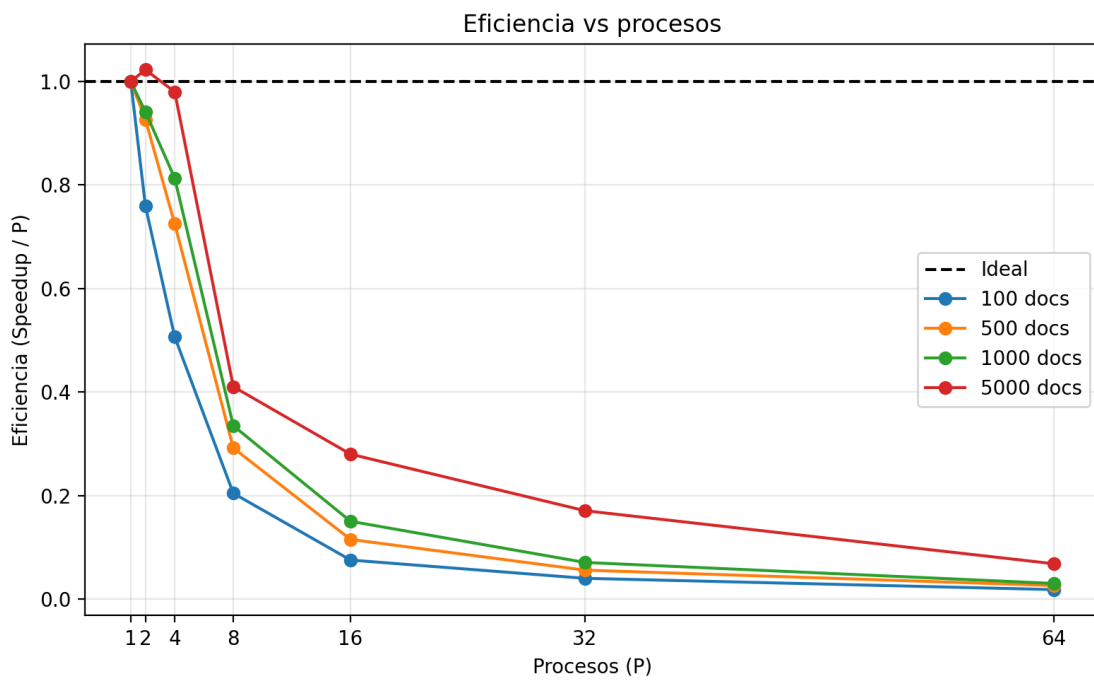


Figure 3: Eficiencia por cantidad de procesos.

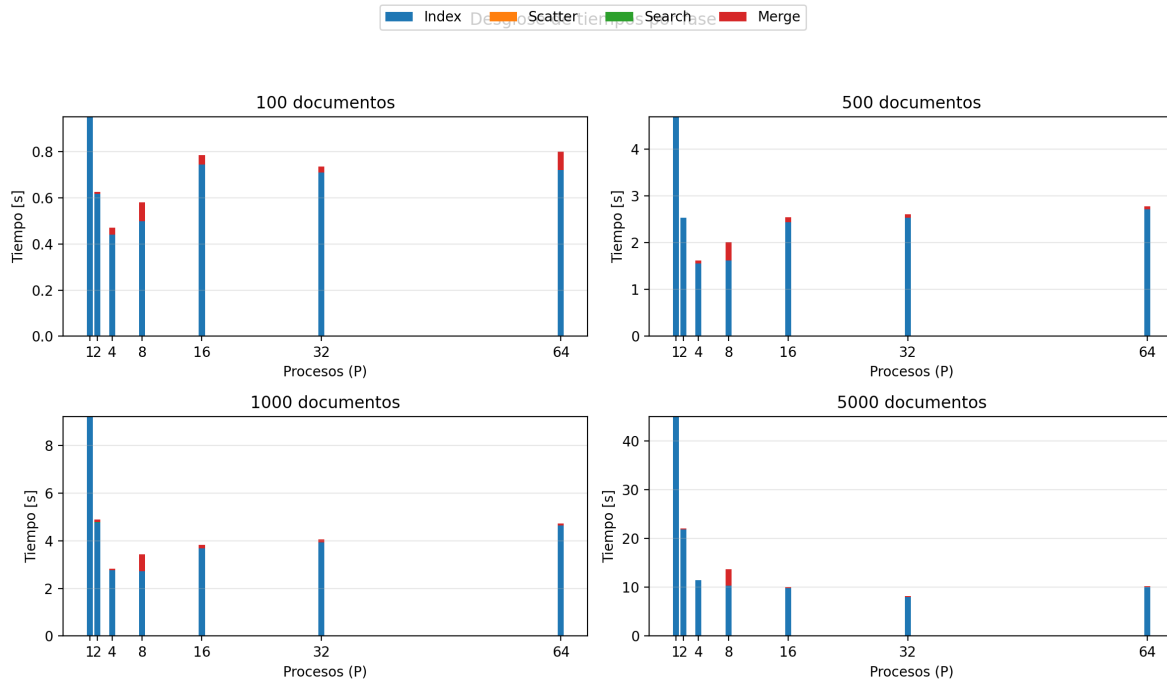


Figure 4: Desglose de tiempos por fase (Index/Scatter/Search/Merge).

- **Overhead en P altos:** el Gather/Merge de Top-K crece y reduce la eficiencia cuando P es muy grande.
- **Mejor punto operativo:** para 5000 documentos, P=16–32 ofrece el mejor balance entre speedup y eficiencia.

10 Conclusiones

La version paralelizada de TF-IDF e IDF mejora claramente el rendimiento y reduce el trafico MPI, eliminando el **Scatter** de matrices grandes. El cuello de botella se desplaza a la recoleccion y fusion de Top-K para P altos. Una mejora futura seria hacer un merge jerarquico o parcial para reducir el costo del **Gather**.

11 Como reproducir las graficas

Ejecutar los scripts desde la raiz del proyecto:

```
python3 plot_scaling.py
python3 plot_breakdown.py
```

Las figuras se guardan en el directorio **figuras/**.